VYBE — Architecture

Overview

VYBE is a single-process Flutter application with no backend. All data lives on the device. The architecture follows a unidirectional data flow: user interaction → provider mutation → UI rebuild. There are no controllers, blocs, or ViewModels — Riverpod providers are the entire state layer.

```
┌──────────────────────────────────────────────────────────┐
│                          UI Layer                        │
│  Screens → ConsumerWidgets watching Riverpod             │
└──────────────────────────┬───────────────────────────────┘
                           │ watch / read
┌──────────────────────────▼───────────────────────────────┐
│                     Provider Layer                       │
│  StreamProviders, FutureProviders, StateProviders        │
└──────────────┬──────────────────────┬────────────────────┘
               │                      │
┌──────────────▼──────────┐  ┌────────▼──────────────────┐
│   VybeAudioEngine       │  │  Repositories             │
│   (just_audio)          │  │  PlaylistRepo (Hive)      │
│   BehaviorSubject       │  │  LyricsRepo (HTTP)        │
│   stateStream           │  │  VybePersistence (prefs)  │
└──────────────┬──────────┘  └───────────────────────────┘
               │
┌──────────────▼───────────────────────────────────────────┐
│                 Android Platform Channels                │
│  HiResAudioChannel / AudioEffectsChannel /               │
│  BitPerfectChannel  (Kotlin)                             │
└──────────────────────────────────────────────────────────┘
```

Data Flow

Playing a song

```
User taps song tile
  → _SongTile.onTap
  → engine.loadQueue(tracks, startIndex: i)
      → _tryBuildSource() for each track  (skips invalid URIs)
      → _playlist.clear() + addAll()
      → _player.seek(Duration.zero, index: i)
      → _player.play()
      → ExoPlayer begins decoding
```

```
  → _player.playerStateStream emits new PlayerState
  → Rx.combineLatest5 assembles VybePlayerState
  → BehaviorSubject<VybePlayerState> emits
  → playerStateProvider (StreamProvider) rebuilds
  → currentTrackProvider, isPlayingProvider, etc. rebuild
  → MiniPlayer and NowPlayingScreen repaint
```

Permission grant → library load

```
main.dart (bootstrap)
  → Permission.audio.request()
  → appContainer.read(permissionGrantedProvider.notifier).state = true
  → localTracksProvider watches permissionGrantedProvider
  → re-runs querySongs()
  → filteredSongsProvider rebuilds
  → _SongsTab repaints with song list
```

DSP mode switch

```
User taps engine card in Settings
  → ref.read(dspModeProvider.notifier).state = DspMode.enhanced
  → engine.setDspMode(DspMode.enhanced)
      → _persistence.saveDspMode(mode)
      → _applyDspNow(mode)
          → ramp _player.setVolume 1.0 → 0.0 over 120 ms
          → _applyEnhancedEq()           (writes band gains)
          → _loudnessEnhancer.setEnabled(true)
          → ramp _player.setVolume 0.0 → 1.0 over 120 ms
  → UI reflects new selection immediately (StateProvider)
```

Provider Dependency Graph

```
permissionGrantedProvider (StateProvider<bool>)
  └── localTracksProvider  (FutureProvider)
  └── localAlbumsProvider  (FutureProvider)
  └── localArtistsProvider (FutureProvider)
        └── filteredSongsProvider (Provider — derived)

audioEngineProvider (Provider<VybeAudioEngine>)
  └── playerStateProvider (StreamProvider<VybePlayerState>)
        ├── currentTrackProvider      (Provider<Track?>)
        ├── isPlayingProvider         (Provider<bool>)
        ├── playerPositionProvider    (Provider<Duration>)
```

```
        ├── playerDurationProvider    (Provider<Duration?>)
        ├── playbackProgressProvider  (Provider<double>)
        ├── queueProvider             (Provider<List<Track>>)
        └── activeTierProvider        (Provider<PlaybackTier>)

currentTrackProvider
  └── dynamicPaletteProvider (AsyncNotifierProvider<DynamicPalette>)
        → extracts colours from album art
        → used by NowPlayingScreen background + MiniPlayer border

lyricsRepoProvider (Provider<LyricsRepository>)
  ├── watches geniusApiKeyProvider
  └── watches anthropicApiKeyProvider
        └── lyricsProvider (FutureProvider.family<Lyrics, Track>)

playlistRepoProvider (Provider<PlaylistRepository>)
  └── playlistsProvider (StreamProvider<List<VybePlaylist>>)
        → streams Hive box listenable → reactive playlist list
```

Audio Engine

`VybeAudioEngine` owns a single `AudioPlayer` instance for the lifetime of the app. It is created in `app.dart` via `WidgetsBinding.addPostFrameCallback` (never before the first frame — avoids cold-start jank).

Pipeline architecture

```
File URI
  → ExoPlayer decoder (FLAC / MP3 / AAC / OGG)
  → AudioPipeline
      ├── AndroidEqualizer      (10-band IIR, disabled in Transparent)
      └── AndroidLoudnessEnhancer (gain trim, disabled in Transparent)
  → AudioTrack
  → Android Audio HAL
  → DAC → headphones / speaker
```

Both `AndroidEqualizer` and `AndroidLoudnessEnhancer` are inside the ExoPlayer `AudioPipeline`. They operate on the decoded float PCM before it reaches `AudioTrack`. This means:

- They are bounded by Android's software volume stack — no overdriving possible.
- They are per-session, not system-wide — they do not affect other apps.
- They require the `AudioPipeline` to be set at player construction time and cannot be changed after.

Stall watchdog

ExoPlayer occasionally stalls at malformed VBR MP3 frame boundaries. The decoder stops producing audio but reports `ProcessingState.ready` — so the player state looks healthy. The watchdog (`Timer.periodic`, 1500 ms) detects this by comparing the current position to the position from the previous tick. If they are equal while the player reports it is playing and ready, it seeks forward 200 ms to skip the damaged frame. The seek is inaudible at normal listening levels.

DSP transition

`setEnabled()` on Android `AudioEffect` subclasses is an instantaneous hardware register write — calling it on a live signal produces an audible click. The smooth transition works by fading the ExoPlayer software volume (`AudioPlayer.setVolume`) to zero first (12 steps × 10 ms = 120 ms), toggling the effect state at silence, then fading back up. Total duration: 240 ms. The user perceives a very brief dip identical to Tidal's engine switch behaviour.

Storage

Hive

Two typed boxes:

Box	Model	typeId	Contents
`tracks`	`Track`	0	Not used for persistence currently — tracks are queried live from `on_audio_query`
`playlists`	`VybePlaylist`	1	User-created playlists with ordered track ID lists

`VybePlaylist.trackIds` stores `'local_{songId}'` strings. When loading a playlist for playback, `PlaylistDetailScreen` joins these IDs against the live `localTracksProvider` result to build `Track` objects. This means playlists survive file moves as long as the song ID (assigned by Android's MediaStore) stays the same.

SharedPreferences

`VybePersistence` stores four keys:

Key	Type	Default
`vybe_loop_mode`	String (`off`/`all`/`one`)	`off`
`vybe_shuffle`	bool	`false`
`vybe_dsp_mode`	String (`transparent`/`enhanced`)	`transparent`
`vybe_playback_playing`	bool	`false`

Dynamic Palette

`DynamicPaletteNotifier` is an `AsyncNotifier` that watches `currentTrackProvider`. When the track ID changes it runs palette extraction on a background isolate (palette_generator uses compute internally). Results are cached in a static `Map<String, DynamicPalette>` (max 30 entries, FIFO eviction).

Two colours are derived:

Field	Source	Use
`vibrant`	`lightVibrantColor ?? vibrantColor ?? lightMutedColor ?? dominantColor`	Glow shadow, mini player border, progress bar, seek bar
`tint`	`vibrant` at 25% HSL lightness, +15% saturation	Background overlay at alpha 110 in Now Playing

Fallback for both is VYBE pink (`#FF1B6B`) — the UI never blocks waiting for extraction.

Lyrics Pipeline

`LyricsRepository.fetchLyrics()` runs three sources in sequence, stopping at the first non-empty result:

```
1. LRCLIB /api/get    (exact match: title + artist + album)
2. LRCLIB /api/search (fuzzy: title + artist, prefers synced results)
3. Genius API         (plain text, requires API key in Settings)
4. Anthropic API      (generated fallback, requires API key in Settings)
```

LRCLIB returns LRC-format synced lyrics (timestamps per line). These power the auto-scrolling karaoke display. The binary search in `_LyricsPanelState` finds the current line in O(log n).

Title cleaning strips common YouTube suffixes before querying (`(Official Music Video)`, `(feat. …)`, `[4K]`, `VEVO`, `- Topic`, etc.) to maximise match rate.

Navigation

VYBE uses a single `MaterialApp` with no named routes. Navigation is managed by two mechanisms:

1. **Bottom nav index** — `bottomNavIndexProvider` (StateProvider<int>) controls which of the three root screens is visible. This is a simple array index — screens are never pushed onto a Navigator stack.
2. **Full-screen player overlay** — `playerExpandedProvider` (StateProvider<bool>) controls a `Positioned.fill` overlay in `HomeScreen`. The player is always in the widget tree when expanded; it slides up from `Offset(0, 1)` via a `SlideTransition`.
3. **Detail screens** — `AlbumDetailScreen` and `ArtistDetailScreen` are pushed with `Navigator.of(context).push(MaterialPageRoute(...))`. Back navigation uses the standard Android back stack.

`PopScope(canPop: false)` in `HomeScreen` intercepts the Android back gesture: if the player is expanded it collapses it; otherwise it calls `SystemNavigator.pop()` to exit.

Threading Model

Work	Thread
UI rendering	Flutter main isolate
Audio decoding	ExoPlayer's internal threads (C++)
Hive reads/writes	Main isolate (Hive is synchronous, fast enough)
Lyrics HTTP requests	Dart async (event loop, no blocking)
Palette extraction	Background compute isolate (palette_generator)
Platform channel calls	Kotlin main thread (marshalled by Flutter)

There are no explicit `Isolate.spawn` calls in application code. All async work uses `Future`/`async-await` on the event loop.